

Stoquify Agent Runtime

Strategic Roadmap Report

Executed roadmap prompt | 2026-07-21

Generated: 2026-07-21

Source prompt:

docs/agents-runtime/STOQUIFY_AGENT_RUNTIME_ROADMAP_REFINED_PROMPT_2026-07-21.md

Source report: docs/agents-runtime/STOQUIFY_AGENT_RUNTIME_PROPOSAL_REPORT_2026-07-21.md

Scope: strategic, architectural, and implementation roadmap only. No application code or schema changes were made.

Executive Summary

Stoquify should implement the agent-runtime proposal as a controlled operating copilot program, not as a generic AI chatbot feature. The implementation should be a shared, Stoquify-owned runtime that sits above the existing source-of-truth services and below the user-facing command surfaces. Its job is to translate trusted business data into role-specific explanations, evidence-backed recommendations, action drafts, approval routes, and safe workflow execution.

The highest-leverage path is deliberately narrow:

1. Add one shared agent runtime.
2. Wrap existing Stoquify services as permission-filtered, module-entitled, redacted tools.
3. Launch one read-only Command Agent first.
4. Add Cash/Reconciliation and Inventory/Replenishment agents in read-and-draft mode.
5. Add approval, idempotency, durable execution, observability, and low-risk controlled actions only after usage proves the workflows.

This roadmap positions Stoquify as a daily operating intelligence layer for African and OHADA SMBs. The destination is not "dashboards plus AI." It is a trusted command system that tells each role what matters now, why it matters, what evidence supports it, who can resolve it, and which controlled workflow should be used.

Evidence Basis

This roadmap is grounded in the generated proposal report and current workspace evidence.

Existing Stoquify strengths to reuse:

- Protected action boundary: services/_shared/protect.ts
- RBAC and audit decisions: lib/security/rbac.ts
- Module entitlement: services/modules/module-entitlement.service.ts
- Module contracts: services/modules/module-control-contracts.ts
- Business signals and action queues: services/signals
- Tenant, payment, inventory, and close snapshots: services/snapshots
- Payment reconciliation workbench:
services/payments/payment-reconciliation-workbench.service.ts
- Inventory cash truth: services/snapshots/inventory-cash-snapshot.service.ts
- Stock-to-cash contracts: services/stock-to-cash
- Payroll command read model: services/payroll/command-read-model.service.ts
- Close assurance: services/accounting/close-assurance.service.ts
- Ledger posting and idempotency: services/accounting/posting.service.ts
- Step-up auth and redaction: services/security
- Evidence contracts and proof drawer: services/evidence, components/evidence
- Command, daily digest, finance, inventory, payroll, compliance, and assurance routes under
app/[locale]/(dashboard)/dashboard

Current implementation gaps observed:

- services/agents does not exist.
- components/agents does not exist.
- app/[locale]/(dashboard)/dashboard/agent-runs does not exist.
- No Prisma agent runtime models were found for AgentRun, AgentStep, AgentApproval, AgentActionDraft, or equivalent.
- No AI runtime packages were found in package.json or package-lock.json for AI SDK, Inngest, Trigger.dev, Langfuse, Helicone, LiteLLM, or MCP.

Existing verification scripts to preserve and extend:

- npm run prisma:validate
- npm run typecheck
- npm run lint
- npm run service:boundary:fail
- npm run workflow:assurance:runtime-check
- npm run inventory:boundary:fail
- npm run payment:cash-truth:gate
- npm run ledger:close-truth:gate
- npm run report:trust:export:gate
- npm run verify:repo

External technology references from the source report:

- Vercel AI SDK: <https://vercel.com/blog/ai-sdk-7>, <https://ai-sdk.dev/docs/ai-sdk-core/tools-and-tool-calling>
- Model Context Protocol: <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- Inngest: <https://www.inngest.com/docs/learn/inngest-functions>
- Trigger.dev: <https://trigger.dev/docs/tasks/overview>
- Langfuse: <https://langfuse.com/?tab=observability>
- Helicone: <https://docs.helicone.ai/getting-started/platform-overview>
- LiteLLM: <https://docs.litellm.ai/>

1. Strategic North Star

Stoquify should become the operating copilot for SMB work, with OHADA-aware evidence and controlled execution as the differentiator.

The product destination:

- For owners: a daily command view that explains cash, stock, risk, and what needs approval.
- For managers: a work queue that turns signals into assigned, evidence-backed actions.
- For finance officers: a cash and reconciliation assistant that explains unmatched money, duplicate references, suspense risk, and cash exposure.
- For accountants: a close and ledger readiness assistant that explains source links, posting state, close blockers, and evidence quality.
- For stockkeepers and purchasing users: an inventory and replenishment assistant that explains stockout, overstock, transfer delay, supplier, and cash-tied-in-stock risk.
- For HR and payroll specialists: a readiness assistant that explains missing employee data, contracts, attendance, payment destination, declarations, redaction, and close exposure.
- For POS cashiers and supervisors: an operating assistant that explains drawer variance, payment completion, refund/void risk, and end-of-day blockers.
- For auditors: a proof trail and evidence assistant that explains what is certified, reconciled, posted, operational, raw, blocked, or redacted.

Why this is defensible:

- Generic AI tools do not own Stoquify's tenant-scoped source data.
- Generic ERP tools rarely expose evidence-grade continuity from POS to cash to inventory to

ledger to close.

- Generic SMB dashboards show metrics, but do not route work with proof, permissions, redaction, approvals, and workflow state.
- Generic chatbots produce answers, but Stoquify can produce controlled actions tied to business records and service-owned workflows.

The new category should be described as:

> Stoquify is the evidence-backed command system for SMB operators, finance teams, accountants, inventory teams, payroll teams, and managers.

That positioning is stronger than "business management SaaS" because it gives Stoquify daily and hourly relevance. It is stronger than "AI copilot" because the value comes from trusted operational truth, not the model itself.

2. Synthesized Solution Architecture

The full solution should be one integrated system with eight layers.

User surfaces

Command panels, module assistant panels, evidence drawers, action draft cards, approval timelines, exception queues, agent-run admin views

Agent runtime

Agent definitions, runs, steps, skills, tools, evidence links, action drafts, approvals, feedback, cost ledger, policy incidents

Agent skills

Trusted context, permission guard, evidence retrieval, redaction, freshness, action drafting, approval routing, trust monitoring

Tool registry

Service-owned read tools and controlled action-draft tools only

Stoquify source-of-truth services

Snapshots, business signals, action queue, proof trails, payment reconciliation, inventory cash truth, stock-to-cash, close assurance, payroll command read model

Control plane

RBAC, module entitlement, fresh auth, maker-checker, redaction, safe errors, idempotency, audit logging, service boundary checks

Optional execution and observability

AI SDK tool loop, MCP-compatible tool contracts, one durable executor, redacted LLM traces, model routing, cost tracking, evaluations

Release assurance

Agent-specific tests and gates plus existing Stoquify policy gates

The runtime must not own business truth. It owns orchestration metadata. Source-of-truth remains in existing domain services.

Primary runtime responsibilities:

- Resolve trusted context for the current user, tenant, role, locale, currency, module access, and time window.
- Select only tools that the actor can use.
- Pull redacted, evidence-bound read-model data from existing services.
- Produce explanations and recommendations with source evidence.
- Create action drafts for human review.
- Route approval and fresh-auth requirements.
- Record runs, steps, costs, feedback, and policy incidents.
- Reject unsafe or unregistered tool requests.

This architecture lets Stoquify move fast without creating a parallel ERP. The agent runtime becomes a thin, auditable layer over protected services.

3. Current-State Fit

Stoquify is unusually ready for this roadmap because the hard substrate already exists.

services/_shared/protect.ts already shows the right pattern: permission requirement, optional fresh auth, tenant guard, module gate, safe errors, correlation IDs, and module access observation. Agent tools should be thin wrappers around this pattern.

lib/security/rbac.ts already establishes session-backed RBAC, active organization enforcement, stale session organization detection, permission expansion, and audited allowed/denied decisions. The agent context resolver should use this rather than implementing its own authorization.

services/modules already has a canonical module catalog and entitlement decision model. Agent visibility must respect module slug, surface type, access intent, and observe/enforce mode.

services/signals already has the strongest first abstraction for the Command Agent. Business signals include severity, required permission, assigned role, action path, evidence grade, blockers, redactions, source hash, and proof link. The agent should summarize and explain these signals.

services/snapshots already provides structured operating truth. Snapshot contracts include freshness, source hash, evidence grade, blockers, redactions, source modules, and UI state. That is almost exactly the kind of data the agent needs.

services/payments already supports payment reconciliation workbench data with rails, failures, duplicate provider references, suspense readiness, and current persistence limitations. This is the seed of the Cash/Reconciliation Agent.

services/snapshots/inventory-cash-snapshot.service.ts already detects zero and negative stock and links inventory to cash and accounting source modules. This is the seed of the Inventory/Replenishment Agent.

services/payroll/command-read-model.service.ts is a later expansion path, not an MVP dependency. It already contains role scope, redaction, country-pack evidence, readiness, blockers, next actions, and release readiness. That makes payroll a high-value phase after the runtime is trusted.

services/accounting/close-assurance.service.ts and services/accounting/posting.service.ts define the sensitive boundary. Agents must not post ledgers or certify close packs. They should explain close blockers and draft actions routed through existing services.

components/evidence/ProofTrailDrawer.tsx already gives a UI pattern for showing evidence chains, blockers, redactions, and audit status. Agent answers should reuse this mental model.

config/sidebar.ts already groups the product into Command, Operations, Finance & Trust, People, and Governance. Agent UI should map into those sections rather than adding a disconnected "AI" area.

4. Implementation Principles

These principles should be treated as release-gate requirements.

1. One shared runtime, not one framework per module.
2. Service-owned tools only.
3. No raw Prisma tool.
4. No direct database writes by agents.
5. No direct ledger posting by agents.
6. No direct statutory filing by agents.
7. No payroll approval, payroll release, or payroll mutation by agents.

8. No role, permission, entitlement, or security setting changes by agents.
9. No close certification or close waiver approval by agents.
10. No direct cash drawer adjustment by agents.
11. No stock adjustment, stock write-off, stock count approval, or purchase order approval by agents.
12. Evidence, freshness, and redaction must be resolved before model output.
13. Agent recommendations must link to source evidence or explicitly say evidence is unavailable.
14. Action drafts must be validated server-side before display and again before approval replay.
15. Human approval is required for medium, high, critical, financial, payroll, compliance, or governance actions.
16. Durable execution must be idempotent.
17. Agent traces must not store unredacted sensitive payloads.
18. Agent UX must be workflow-first, role-aware, and embedded in existing Stoquify surfaces.

5. Phased Roadmap

Phase 0: Readiness And Design Freeze

Objective:

- Freeze the agent runtime boundaries before writing code.
- Confirm what is in scope for MVP and what remains prohibited.

Technical work:

- Create an architecture decision record under docs/agents-runtime.
- Define agent risk levels: read-only, draft, low-risk action, sensitive action, prohibited.
- Define tool categories: read tool, explanation tool, action draft tool, approval tool, blocked tool.
- Define data classification for model context.
- Define redaction-before-prompt policy.
- Define first pilot roles: owner, manager, accountant, finance officer, stockkeeper.

Files/modules likely affected:

- docs/agents-runtime
- optional what-next/agents-runtime
- no application code

Database changes:

- None.

UI surfaces:

- None.

Tests:

- None required, but produce an implementation checklist and risk register.

Risks:

- Scope creep into generic chatbot work.
- Premature dependency adoption.

Dependencies:

- Product/engineering/security agreement on prohibited actions.

Acceptance criteria:

- Written design freeze exists.
- Prohibited tool list is approved.
- MVP agents and tools are selected.
- Dependency adoption order is approved.

Release gates:

- Document review only.

Phase 1: Shared Runtime Foundation

Objective:

- Add the minimum runtime needed to record agent runs, tools, steps, evidence, feedback, and policy incidents.

Technical work:

- Add Prisma models for AgentDefinition, AgentSkillDefinition, AgentToolDefinition, AgentRun, AgentStep, AgentEvidenceLink, AgentFeedback, AgentCostLedger, and AgentPolicyIncident.
- Add services/agents/agent-contracts.ts.
- Add agent-context.service.ts using RBAC, module entitlement, locale, currency, tenant, and route context.
- Add agent-tool-registry.service.ts with hardcoded MVP read tools.
- Add agent-evidence.service.ts to bind evidence grade, source hash, freshness, blockers, redactions, and proof subjects.
- Add agent-redaction.service.ts over existing redaction policies.
- Add agent-runner.service.ts as a deterministic non-LLM stub first.
- Add tests proving no raw Prisma or direct write tool is registered.

Files/modules likely affected:

- prisma/schema.prisma
- services/agents
- services/agents/__tests__
- optional seed file for default agent definitions

Database changes:

- Introduce runtime tables but no business data mutation path.

UI surfaces:

- None required beyond optional admin/dev inspection later.

Tests:

- agent-context.service.test.ts
- agent-tool-registry.service.test.ts
- agent-evidence.service.test.ts
- agent-redaction.service.test.ts
- agent-no-direct-write-boundary.test.ts

Risks:

- Runtime models become too broad.
- Tool registry accidentally exposes unsafe service paths.

Dependencies:

- Existing RBAC, module entitlement, snapshots, signals, evidence, and redaction services.

Acceptance criteria:

- Runtime tables validate through Prisma.

- All registered tools have module slug, required permission, risk level, output schema, and evidence behavior.
- No direct Prisma write tool exists.
- Agent runs can be recorded with steps and evidence links.

Release gates:

- npm run prisma:validate
- npm run typecheck
- npm run lint
- npm run service:boundary:fail
- focused services/agents Jest tests

Phase 2: Read-Only Command Agent

Objective:

- Launch the first internal agent as a read-only command brief that explains daily priorities using existing Stoquify truth.

Technical work:

- Add AI SDK or equivalent TypeScript-native model/tool loop.
- Register read-only tools:
 - readTenantOperatingSnapshot
 - readBusinessSignals
 - readActionQueue
 - readProofTrail
 - readCloseReadinessSnapshot
- Add prompt templates as versioned agent skills.
- Ensure every response includes evidence grade, freshness, source modules, and limitations.
- Add feedback capture.

Files/modules likely affected:

- services/agents
- components/agents/AgentCommandPanel.tsx
- components/agents/AgentEvidenceDrawer.tsx
- app/[locale]/(dashboard)/dashboard/daily-digest/page.tsx
- app/[locale]/(dashboard)/dashboard/manager-action-center/page.tsx
- app/[locale]/(dashboard)/dashboard/owner-war-room/page.tsx

Database changes:

- Use existing Phase 1 runtime tables.
- No action draft or approval tables yet unless included in Phase 1 as inactive.

UI surfaces:

- Daily Digest agent panel.
- Manager Action Center agent panel.
- Owner War Room command brief.
- Proof trail drawer integration.

Tests:

- Role-specific context tests.
- Permission-filtered signal tests.
- Redaction-before-output tests.
- Stale snapshot response tests.
- UI rendering tests for evidence badges and no-visible-actions state.

Risks:

- Users expect the agent to execute actions.
- Agent hallucination if context is not tightly bounded.

Dependencies:

- AI SDK or equivalent.
- Existing snapshots/signals/action queue/proof trail services.

Acceptance criteria:

- Agent can answer "what matters today" using only allowed context.
- Agent refuses or safely redirects action requests.
- Agent output is evidence-bound.
- Agent output changes by role and permission.
- Agent run history records steps and feedback.

Release gates:

- npm run typecheck
- npm run lint
- npm run workflow:assurance:runtime-check
- npm run report:trust:export:gate
- focused component and service tests

Phase 3: Cash/Reconciliation Agent In Read-And-Draft Mode

Objective:

- Convert payment and cash truth into daily reconciliation triage and draft recommendations.

Technical work:

- Register read tools:
 - readPaymentTruthSnapshot
 - readPaymentReconciliationWorkbench
 - readCashPaymentHistory
 - readProofTrail
- Add draft tools:
 - draftReconciliationMatchSuggestion
 - draftSuspenseClassification
 - draftEvidenceRequest
 - draftActionItemAssignment
- Add AgentActionDraft if not already introduced.
- Add draft validation service.
- Add visible risk labels: read-only, draft only, requires approval, prohibited.

Files/modules likely affected:

- services/agents
- services/payments
- components/agents/AgentActionDraftCard.tsx
- components/finance/PaymentReconciliationWorkbench.tsx
- components/finance/CashPaymentHistoryWorkbench.tsx
- app/[locale]/(dashboard)/dashboard/finance/reconciliation/page.tsx
- app/[locale]/(dashboard)/dashboard/finance/cash-command/page.tsx

Database changes:

- AgentActionDraft
- optional AgentDraftEvidenceLink

UI surfaces:

- Finance reconciliation assistant panel.
- Cash command exception explanation.
- Draft suggestion cards.
- Evidence drawer for reconciliation runs and payment transactions.

Tests:

- Duplicate provider reference triage.
- Suspense suggestion remains draft-only.
- No direct ledger posting.
- No direct cash adjustment.
- Payment provider references masked or redacted by permission.

Risks:

- False match suggestions.
- Users over-trust draft recommendations.

Dependencies:

- Payment reconciliation workbench.
- Payment truth snapshot.
- Redaction policy for payment provider references and suspense details.

Acceptance criteria:

- Agent explains open suspense and duplicate reference risk.
- Agent creates draft suggestions only.
- Drafts include evidence, source hash, required permission, and approval requirement.
- Agent cannot post suspense, reconcile, or mutate cash.

Release gates:

- npm run payment:cash-truth:gate
- npm run ledger:close-truth:gate
- npm run report:trust:export:gate
- focused payment/agent tests

Phase 4: Inventory/Replenishment Agent In Read-And-Draft Mode

Objective:

- Turn inventory cash truth and stock-to-cash continuity into actionable stock risk and replenishment drafts.

Technical work:

- Register read tools:
 - readInventoryCashSnapshot
 - readStockToCashFlow
 - readInventoryMovements
 - readItemSupplierLinks
 - readPurchaseOrderSignals
- Add draft tools:
 - draftReplenishmentSuggestion
 - draftTransferReview
 - draftStockCountReview
 - draftPurchaseOrderReview
- Add velocity and risk scoring as deterministic service logic before model explanation.

Files/modules likely affected:

- services/agents
- services/snapshots/inventory-cash-snapshot.service.ts

- services/stock-to-cash
- inventory hooks/components
- app/[locale]/(dashboard)/dashboard/inventory/page.tsx
- app/[locale]/(dashboard)/dashboard/inventory/items/page.tsx
- app/[locale]/(dashboard)/dashboard/inventory/movements/page.tsx
- app/[locale]/(dashboard)/dashboard/inventory/transfers/page.tsx
- app/[locale]/(dashboard)/dashboard/purchase-orders/page.tsx
- app/[locale]/(dashboard)/dashboard/finance/stock-to-cash/page.tsx

Database changes:

- Reuse AgentActionDraft.
- No direct stock mutation table.

UI surfaces:

- Inventory assistant panel.
- Stock-to-cash risk explanation.
- Replenishment draft card.
- Transfer review draft.
- Count variance review draft.

Tests:

- Negative stock blocker explanation.
- Zero stock risk explanation.
- Draft reorder never creates PO directly.
- Draft transfer never updates inventory.
- Stock write-off is prohibited.

Risks:

- Replenishment suggestions become noisy if demand data is thin.
- Users confuse suggestions with approved purchase orders.

Dependencies:

- Inventory cash snapshot.
- Stock-to-cash service.
- Purchasing and item supplier read models.

Acceptance criteria:

- Agent identifies negative stock, zero stock, dead stock exposure, and replenishment candidates.
- Every suggestion is draft-only and evidence-bound.
- No stock adjustment, write-off, transfer, or PO approval can be executed by the agent.

Release gates:

- npm run inventory:boundary:fail
- npm run service:boundary:fail
- npm run ledger:close-truth:gate
- focused inventory/agent tests

Phase 5: Approval, Idempotency, And Low-Risk Controlled Actions

Objective:

- Allow only approved, low-risk service actions after draft validation, approval, fresh-auth where needed, and idempotency.

Technical work:

- Add AgentApproval.

- Add approval replay validation.
- Add signed input hash or equivalent tamper detection.
- Add maker-checker rules by action risk.
- Add idempotency key generation for action drafts.
- Add low-risk controlled action execution only through protected services.
- Define medium/high/critical actions as approval-only or prohibited.

Allowed early low-risk actions:

- assign an action item
- dismiss an agent suggestion as not relevant
- create an evidence request
- create a reminder or follow-up task
- mark an agent draft as accepted for human follow-up

Still prohibited:

- ledger posting
- cash adjustment
- stock adjustment or write-off
- payroll approval or release
- statutory filing
- close certification
- role or permission change

Files/modules likely affected:

- services/agents/agent-approval.service.ts
- services/agents/agent-action-draft.service.ts
- components/agents/AgentApprovalTimeline.tsx
- services/signals/action-queue.service.ts
- selected protected action wrappers

Database changes:

- AgentApproval
- action execution result fields on AgentActionDraft

UI surfaces:

- Approval timeline.
- Approval modal or side panel.
- Draft replay details.
- Execution result badge.

Tests:

- Approval cannot replay changed input.
- Approval fails if actor loses permission.
- Approval fails if module entitlement changes.
- Approval fails when fresh auth expires.
- Idempotency prevents duplicate service execution.

Risks:

- Approval flows are perceived as slow.
- Users try to use agents as a shortcut around controls.

Dependencies:

- Fresh auth.
- RBAC.
- Module entitlement.
- Safe action responses.

Acceptance criteria:

- Low-risk actions execute through protected service paths.
- Medium/high/critical actions require approval or remain prohibited.
- Approval replay revalidates all controls.
- Every execution has audit and agent history.

Release gates:

- npm run service:boundary:fail
- npm run workflow:assurance:runtime-check
- npm run hard-delete:fail
- focused approval/idempotency tests

Phase 6: Observability, Cost Control, Evaluations, And Trust Monitoring

Objective:

- Make the agent runtime measurable, auditable, and governable in production.

Technical work:

- Add redacted trace integration.
- Add cost ledger aggregation by tenant, agent, module, and model.
- Add prompt/template version tracking.
- Add feedback dashboards.
- Add policy incident dashboard.
- Add stale-answer and unsafe-attempt detection.
- Add evaluation datasets from accepted/rejected recommendations.
- Add one model routing strategy when traffic justifies it.

Technology options:

- Langfuse for trace/evaluation/prompt management emphasis.
- Helicone for gateway, request observability, caching, routing, and cost control emphasis.
- LiteLLM later for multi-provider routing and budgets.

Files/modules likely affected:

- services/agents/agent-observability.service.ts
- app/[locale]/(dashboard)/dashboard/agent-runs/page.tsx
- components/agents
- optional scripts/agent-runtime-gate.js

Database changes:

- Enhance AgentCostLedger.
- Enhance AgentPolicyIncident.
- Add evaluation metadata if not externalized.

UI surfaces:

- Agent run admin surface.
- Agent trust monitor.
- Cost and usage dashboard.
- Policy incident queue.

Tests:

- Trace redaction.
- Prompt version hash stored.
- Cost ledger records model and token metadata.
- Policy incident generated for prohibited tool attempts.

Risks:

- Observability stores sensitive data.
- Cost controls arrive too late.

Dependencies:

- Mature enough agent traffic to make traces useful.
- Redaction policy and privacy review.

Acceptance criteria:

- No unredacted sensitive data appears in traces.
- Costs are visible by tenant/module/agent.
- Policy incidents are reviewable.
- Evaluation loop exists for accepted, rejected, stale, and unsafe outputs.

Release gates:

- npm run report:trust:export:gate
- npm run release:secrets:preflight
- npm run ci:release:gate
- agent trace redaction tests

Phase 7: Expansion Into Purchasing/AP, Close/Compliance, Payroll Readiness, And Customer Success

Objective:

- Expand the runtime only after the first three agents prove trust, adoption, and control.

Expansion agents:

- Purchasing/AP Agent: supplier exposure, PO delays, goods receipt gaps, payables readiness, fraud-control warnings.
- Close/Compliance Agent: close blockers, evidence gaps, waiver routing, statutory readiness, compliance evidence.
- Payroll Readiness Agent: employee, contract, attendance, payment destination, declaration, payroll run, posting, and close exposure.
- Customer Success/Adoption Agent: tenant onboarding, module adoption, stuck workflows, repeated support issues, and usage gaps.

Files/modules likely affected:

- services/purchasing
- services/accounting
- services/compliance
- services/payroll
- services/modules
- components/agents
- corresponding dashboard routes

Database changes:

- No new runtime family if Phase 1 to 6 were designed correctly.
- Add new agent definitions, skills, tools, and evaluation datasets.

UI surfaces:

- AP control workbench.
- Accounting close center.
- Compliance center.
- Payroll command center.
- Settings/modules customer success view.

Tests:

- Payroll redaction and readiness tests.
- Close certification prohibition tests.
- Statutory filing prohibition tests.
- AP fraud-control draft-only tests.
- Customer success tenant isolation tests.

Risks:

- Expansion reaches high-risk domains too early.
- Payroll or statutory output becomes non-authoritative.

Dependencies:

- Proven runtime trust metrics from phases 2 to 6.
- Mature approval and redaction controls.

Acceptance criteria:

- Expansion agents reuse the same runtime.
- No new module-specific agent framework is introduced.
- High-risk actions remain human-owned.
- Evidence and source provenance are mandatory.

Release gates:

- npm run statutory:country-pack:gate
- npm run payroll:immutability:runtime
- npm run ledger:close-truth:gate
- npm run report:trust:export:gate
- domain-specific focused tests

6. Implementation Blueprint

Recommended module layout:

```

services/agents/
  agent-contracts.ts
  agent-context.service.ts
  agent-tool-registry.service.ts
  agent-runner.service.ts
  agent-evidence.service.ts
  agent-redaction.service.ts
  agent-action-draft.service.ts
  agent-approval.service.ts
  agent-observability.service.ts
  agent-policy.service.ts
tools/
  command-tools.ts
  payment-reconciliation-tools.ts
  inventory-replenishment-tools.ts
  proof-trail-tools.ts
skills/
  trusted-context.skill.ts
  permission-entitlement-guard.skill.ts
  evidence-grounded-retrieval.skill.ts
  redaction-safe-output.skill.ts
  freshness-evaluator.skill.ts
  action-draft-builder.skill.ts
  approval-router.skill.ts
  trust-monitor.skill.ts
__tests__/
  agent-context.service.test.ts
  agent-tool-registry.service.test.ts
  agent-evidence.service.test.ts

```

```

agent-redaction.service.test.ts
agent-action-draft.service.test.ts
agent-approval.service.test.ts
agent-no-direct-write-boundary.test.ts
agent-tenant-boundary.test.ts

```

Recommended UI layout:

```

components/agents/
  AgentCommandPanel.tsx
  AgentModuleAssistantPanel.tsx
  AgentEvidenceDrawer.tsx
  AgentActionDraftCard.tsx
  AgentApprovalTimeline.tsx
  AgentRunStatusBadge.tsx
  AgentTrustNotice.tsx
  AgentFeedbackControls.tsx
  AgentPolicyIncidentList.tsx

```

Recommended route additions:

```

app/[locale]/(dashboard)/dashboard/agent-runs/
  page.tsx
  loading.tsx
  error.tsx

app/[locale]/(dashboard)/dashboard/agent-runs/[runId]/
  page.tsx

```

Recommended integration points:

```

app/[locale]/(dashboard)/dashboard/daily-digest/page.tsx
app/[locale]/(dashboard)/dashboard/manager-action-center/page.tsx
app/[locale]/(dashboard)/dashboard/owner-war-room/page.tsx
app/[locale]/(dashboard)/dashboard/finance/reconciliation/page.tsx
app/[locale]/(dashboard)/dashboard/finance/cash-command/page.tsx
app/[locale]/(dashboard)/dashboard/finance/stock-to-cash/page.tsx
app/[locale]/(dashboard)/dashboard/inventory/page.tsx
app/[locale]/(dashboard)/dashboard/inventory/items/page.tsx
app/[locale]/(dashboard)/dashboard/inventory/movements/page.tsx
app/[locale]/(dashboard)/dashboard/inventory/transfers/page.tsx

```

Recommended scripts:

```

scripts/agent-runtime-boundary-gate.js
scripts/agent-tool-registry-gate.js
scripts/agent-redaction-trace-gate.js
scripts/agent-prohibited-action-gate.js

```

Recommended package scripts:

```

{
  "agent:runtime:gate": "node scripts/agent-runtime-boundary-gate.js --mode fail",
  "agent:tools:gate": "node scripts/agent-tool-registry-gate.js --mode fail",
  "agent:redaction:gate": "node scripts/agent-redaction-trace-gate.js --mode fail",
  "agent:prohibited-actions:gate": "node scripts/agent-prohibited-action-gate.js --mode fail"
}

```

These scripts should eventually be added into policy:gates after the runtime is introduced.

7. Data Model Roadmap

Phase 1 Models

AgentDefinition

- Stores canonical agent key, name, owner module, status, rollout mode, risk level, default model policy, allowed tools, and allowed skills.
- Needed to prevent hidden or ad hoc agents.
- Phase: 1.

AgentSkillDefinition

- Stores skill key, version, domain, prompt hash, owner module, required permissions, redaction categories, and evaluation notes.
- Needed to version Stoquify-native expertise.
- Phase: 1.

AgentToolDefinition

- Stores tool key, owner service, module slug, required permission, risk level, input schema hash, output schema hash, approval policy, and idempotency behavior.
- Needed to make model-visible tools auditable and stable.
- Phase: 1.

AgentRun

- Stores tenant, actor, agent, status, source route, correlation ID, started/completed timestamps, model policy, cost estimate, and failure state.
- Needed for traceability and support.
- Phase: 1.

AgentStep

- Stores step order, tool call, prompt hash, input hash, output hash, status, retry count, safe summary, and error category.
- Needed for audit and debugging.
- Phase: 1.

AgentEvidenceLink

- Stores run/step link to proof subject, source module, source table, source hash, evidence grade, freshness, blocker count, and redaction count.
- Needed to make every answer evidence-backed.
- Phase: 1.

AgentFeedback

- Stores actor feedback, accepted/rejected state, stale answer flag, wrong answer flag, unsafe attempt flag, correction text, and follow-up outcome.
- Needed for improvement and trust measurement.
- Phase: 1 or 2.

AgentCostLedger

- Stores model provider, model name, token counts, estimated cost, tenant, module, and budget bucket.
- Needed for cost control.
- Phase: 1 as basic fields, expanded in Phase 6.

AgentPolicyIncident

- Stores prohibited tool attempts, redaction failures, stale context violations, approval replay failures, and unsafe action attempts.
- Needed for governance and release assurance.
- Phase: 1.

Phase 3 To 5 Models

AgentActionDraft

- Stores proposed service action, visible summary, payload hash, required permission, module slug, evidence references, risk level, idempotency key, status, and execution result.
- Needed for read-and-draft agents.
- Phase: 3.

AgentApproval

- Stores draft/action reference, requestedBy, approvedBy, status, approval policy, fresh-auth requirement, signed input hash, expiration, approval time, rejection reason, and replay validation result.
- Needed for controlled action execution.
- Phase: 5.

Optional later model: AgentEvaluationCase

- Stores anonymized/redacted production examples for regression tests and prompt evaluations.
- Needed after real usage exists.
- Phase: 6.

8. Agent And Skill Roadmap

Command Agent

Purpose:

- Turn tenant operating truth, business signals, action queues, and proof trails into role-specific daily priorities.

Users:

- Owner, manager, accountant, finance officer, stockkeeper, HR/payroll lead, auditor.

Tools:

- readTenantOperatingSnapshot
- readBusinessSignals
- readActionQueue
- readProofTrail
- readCloseReadinessSnapshot

Permissions:

- Base dashboard read plus per-signal required permissions.
- Module entitlement by source module.

Risk boundary:

- Read-only in Phase 2.
- Draft-only after Phase 3.

Evidence requirements:

- Evidence grade, freshness, source modules, source hash, blockers, redactions.

UI entry points:

- Daily Digest.
- Manager Action Center.
- Owner War Room.
- dashboard header command launcher.

Success metrics:

- Weekly active users by role.

- Action queue resolution time.
- Recommendation helpfulness.
- Stale answer rate.
- Redaction correctness.

Cash/Reconciliation Agent

Purpose:

- Explain unmatched, duplicated, pending, and suspense payment/cash issues.

Users:

- Finance officer, accountant, owner, manager, cashier supervisor, auditor.

Tools:

- readPaymentTruthSnapshot
- readPaymentReconciliationWorkbench
- readCashPaymentHistory
- readProofTrail
- draftReconciliationMatchSuggestion
- draftSuspenseClassification

Permissions:

- payments.reconciliation.read
- finance payment/cash permissions
- stricter permissions for suspense detail.

Risk boundary:

- No cash adjustment.
- No suspense posting.
- No ledger posting.
- Draft only.

Evidence requirements:

- Reconciliation run, payment transaction, provider reference coverage, suspense amount, evidence grade, redaction state.

UI entry points:

- Finance reconciliation.
- Cash command.
- Cash payment history.
- Finance payments.

Success metrics:

- Open suspense decline.
- Duplicate reference triage time.
- Accepted match suggestion rate.
- False match suggestion rate.

Inventory/Replenishment Agent

Purpose:

- Explain stock risk, cash tied in stock, replenishment priorities, and stock-to-cash impact.

Users:

- Stockkeeper, purchasing officer, manager, owner, cashier supervisor, accountant.

Tools:

- readInventoryCashSnapshot
- readStockToCashFlow
- readInventoryMovements
- readItemSupplierLinks
- draftReplenishmentSuggestion
- draftTransferReview
- draftStockCountReview

Permissions:

- inventory read permissions
- purchasing read permissions for supplier context
- draft permissions for action suggestions.

Risk boundary:

- No stock adjustment.
- No write-off.
- No PO approval.
- No stock count approval.

Evidence requirements:

- stock levels, transaction history, purchase orders, source hash, negative/zero stock blockers.

UI entry points:

- Inventory overview.
- Inventory items.
- Inventory movements.
- Inventory transfers.
- Purchase orders.
- Finance stock-to-cash.

Success metrics:

- Stockout incidents.
- Negative stock blockers.
- Replenishment acceptance.
- Dead stock exposure.

Exception Orchestrator

Purpose:

- Normalize blockers, safe errors, workflow assurance incidents, snapshot partial states, and reconciliation failures into work items.

Users:

- Mostly invisible infrastructure.

Tools:

- read signals
- create internal exception summary
- route to action queue
- attach evidence

Risk boundary:

- No business action execution.

Evidence requirements:

- source module, source hash, blocker, severity, required permission.

UI entry points:

- Assurance Control Tower.
- Manager Action Center.
- Agent Run admin view.

Success metrics:

- Duplicate exception reduction.
- Time to assignment.
- Critical exception visibility.

Action Orchestrator

Purpose:

- Convert accepted recommendations into validated drafts, approvals, and low-risk service executions.

Users:

- Mostly infrastructure, visible through action draft cards and approval timelines.

Tools:

- validate draft
- request approval
- replay approval
- execute allowed service action
- record result

Risk boundary:

- Service-owned execution only.
- Medium/high/critical actions require approval or remain prohibited.

Evidence requirements:

- input hash, approval policy, permission, module entitlement, evidence links, idempotency key.

UI entry points:

- action draft cards
- approval timeline
- agent run detail

Success metrics:

- Approval completion rate.
- Replay validation failures.
- Duplicate execution prevention.
- Audit completeness.

Platform Skills

Trusted Context Resolver

- Resolves user, tenant, org, locale, currency, role, permissions, module entitlements, route, and time window.

Permission and Entitlement Guard

- Filters tools, records, fields, and actions before model context exists.

Evidence-Grounded Retrieval

- Pulls only service-owned read models and attaches proof links, source hashes, evidence grades, blockers, and redactions.

Redaction and Safe Output

- Applies redaction categories before prompts and before final output.

Freshness Evaluator

- Marks answers fresh, stale, partial, blocked, empty, or failed.

Action Draft Builder

- Creates draft payloads with visible summary, permission, risk level, evidence, idempotency key, and prohibited-action checks.

Approval Router

- Determines approver role, fresh-auth requirement, maker-checker policy, expiration, and replay validation.

Agent Trust Monitor

- Watches stale answers, unsafe attempts, cost spikes, low acceptance, policy denials, and repeated tool failures.

9. UI/UX Roadmap

The agent experience should appear as operating intelligence embedded into existing workflows.

Primary UI components:

- AgentCommandPanel: brief, role-aware daily explanation with linked evidence.
- AgentModuleAssistantPanel: contextual panel for "explain this", "what is blocked", and "what should happen next".
- AgentEvidenceDrawer: wrapper around the proof trail drawer with agent run context.
- AgentActionDraftCard: shows recommendation, affected records, evidence, risk, permission, and approval state.
- AgentApprovalTimeline: shows requested, pending, approved, rejected, executed, failed, or expired.
- AgentTrustNotice: shows freshness, redaction, limitations, and source modules.
- AgentFeedbackControls: useful, wrong, stale, unsafe, accepted, rejected.
- AgentRunStatusBadge: visible state for running, completed, failed, blocked, approval required.

Placement:

- Daily Digest: Command Agent daily brief and top priorities.
- Owner War Room: cash, stock, close, and payroll exposure summary.
- Manager Action Center: exception queue and assignment recommendations.
- Finance Reconciliation: Cash/Reconciliation Agent panel and draft cards.
- Finance Cash Command: cash exposure and suspense explanation.
- Inventory Overview: Inventory/Replenishment Agent risk summary.
- Stock-to-Cash: cross-module explanation and evidence.
- Assurance Control Tower: policy incidents and exception orchestration.
- Payroll Command Center: later payroll readiness assistant.

UX rules:

- Do not make a generic chat-first screen.
- Do not hide evidence behind vague generated text.
- Do not show action buttons without permission and risk state.
- Do not show sensitive fields without redaction evaluation.
- Do not let an agent response be the only record of a decision.
- Use badges for evidence grade, freshness, redaction, and approval status.
- Use drawers and side panels for details, not oversized marketing-style surfaces.

10. Technology Adoption Plan

Adopt Immediately: AI SDK Or Equivalent TypeScript Tool Loop

Reason:

- Stoquify is already TypeScript/Next.js.
- Tool calling and structured output map naturally to service-owned tools.
- Streaming can improve command panels.
- Tool lifecycle events can map to AgentStep.

Condition:

- Adopt only after Phase 1 tool registry and no-direct-write gates exist.

Design Immediately: MCP-Compatible Tool Contracts

Reason:

- MCP-style tool contracts create clean names, descriptions, and input schemas.
- Future internal or external agent clients can reuse the boundary.

Condition:

- Do not expose a broad MCP server in MVP.
- Start with internal registry shapes compatible with MCP.

Defer Until Phase 5: Durable Execution

Candidate:

- Inngest or Trigger.dev, not both.

Reason:

- Approval wait, retry, resume, delayed actions, and scheduled digests benefit from durable execution.

Condition:

- Adopt only after idempotency and approval replay rules exist.

Add In Phase 6: Observability And Evaluation

Candidate:

- Langfuse if evaluation, prompt management, and trace quality are most important.
- Helicone if gateway, cost, cache, and routing are most important.
- LiteLLM later if multi-provider routing, budgets, and provider abstraction become important.

Condition:

- No unredacted payroll, bank, fiscal, compliance, close-certification, or audit payloads in

external traces.

- Self-host or configure privacy controls if sensitive trace storage is needed.

Avoid

- Multiple agent frameworks per module.
- Python-first agent framework as the core runtime.
- Raw database tools.
- Browser automation as a business execution layer.
- Agent-visible tools that ignore RBAC or module entitlement.

11. Risk And Control Matrix

Risk	Severity	Owner	Mitigation	Gate	Rollback
Parallel business logic	High	Architecture	Tools wrap service-owned read models only	service boundary gate	disable tool definition
Direct database mutation	Critical	Security/Architecture	no raw Prisma tools, registry denylist	agent prohibited action gate	disable runtime
Ledger posting by agent	Critical	Finance/Accounting	ledger tools prohibited, drafts only	ledger close truth gate	revoke tool
Statutory filing by agent	Critical	Compliance	filing prohibited, evidence review only	statutory country-pack gate	disable compliance agent
Payroll sensitive leakage	Critical	Payroll/Security	redaction before prompt/output	redaction trace gate	disable payroll tools
Permission bypass	Critical	Security	RBAC plus module entitlement before tools	RBAC tests	disable agent route
Stale recommendations	High	Product/Architecture	freshness evaluator, stale labels	snapshot tests	hide recommendation
False match suggestion	High	Finance	draft-only, confidence, human approval	payment cash truth gate	disable draft type
Unsafe approval replay	Critical	Security	signed input hash, revalidate permission/module/fresh auth	approval tests	expire approvals
Cost spike	Medium	Platform	cost ledger, budgets, model policy	cost monitor	downgrade/disable model
Bad UX adoption	Medium	Product/UX	embedded panels, role workflows	route smoke tests	hide panel
Observability data leak	Critical	Security	redacted traces, self-host option	redaction trace gate	disable trace export
Dependency sprawl	Medium	Architecture	one durable layer, one primary observability path	ADR review	remove dependency
Generic chatbot drift	Medium	Product	no standalone chatbot MVP	product review	remove chat entry
OHADA non-authoritative advice	High	Compliance	country-pack provenance and expert review	statutory gate	block answer

12. Verification And Release Plan

Baseline commands:

```
npm run prisma:validate
npm run typecheck
npm run lint
npm run service:boundary:fail
npm run workflow:assurance:runtime-check
npm run inventory:boundary:fail
npm run payment:cash-truth:gate
npm run ledger:close-truth:gate
npm run report:trust:export:gate
```

```
npm test -- --runInBand services/agents
```

Phase 1 verification:

- Prisma validates agent runtime schema.
- Agent tool registry has no raw Prisma tool.
- Every registered tool has owner service, permission, module slug, risk level, and schema hash.
- Agent context differs by tenant and role.
- Redaction tests pass before prompt construction.

Phase 2 verification:

- Command Agent is read-only.
- Role-specific daily brief shows only visible signals.
- Stale snapshots produce caution state.
- Proof trail links render where available.
- Feedback writes only to agent feedback.

Phase 3 verification:

- Cash/Reconciliation Agent cannot mutate reconciliation, cash, or ledger records.
- Draft suggestions include evidence and required permission.
- Suspense details are masked or redacted for unauthorized users.
- Payment cash truth gate passes.

Phase 4 verification:

- Inventory/Replenishment Agent cannot adjust stock, write off stock, approve counts, or approve POs.
- Negative stock and zero stock explanations are evidence-bound.
- Replenishment remains a draft recommendation.
- Inventory boundary gate passes.

Phase 5 verification:

- Approval replay revalidates actor permission, module entitlement, fresh auth, and input hash.
- Idempotency prevents duplicate low-risk action execution.
- Medium/high/critical actions remain approval-only or prohibited.

Phase 6 verification:

- Trace export stores only redacted safe summaries.
- Cost ledger records model, tenant, module, and usage metadata.
- Policy incidents are generated and visible.
- Evaluation loop includes accepted/rejected/stale/unsafe feedback.

Manual role smoke tests:

- Owner sees cross-module priorities.
- Accountant sees close, ledger, reconciliation, and evidence.
- Finance officer sees cash and reconciliation details allowed by permission.
- Stockkeeper sees inventory actions but not payroll or finance sensitive details.
- HR/payroll user sees payroll readiness only when phase 7 is enabled.
- Auditor sees proof and audit context according to permissions.
- Cashier sees POS/cashier-safe operational guidance only.

13. 30/60/90-Day Execution Plan

Days 1-7

Engineering outputs:

- Design freeze document.
- Agent risk taxonomy.
- Prohibited tool denylist.
- Initial Prisma model draft.

Product outputs:

- Pilot role list.
- MVP use cases for Command Agent.
- "What matters today" brief template.

Review checkpoint:

- Architecture/security review.

Decision gate:

- Approve one shared runtime and no-direct-write boundary.

Days 8-14

Engineering outputs:

- services/agents/agent-contracts.ts
- context resolver
- tool registry skeleton
- no-direct-write tests
- evidence binder skeleton

Product outputs:

- Command Agent answer examples.
- Evidence and freshness display rules.

Review checkpoint:

- Service boundary review.

Decision gate:

- Approve MVP read tools.

Days 15-21

Engineering outputs:

- Runtime Prisma migration.
- Agent run/step/evidence persistence.
- Redaction wrapper.
- MVP read tools for snapshots, signals, action queue, proof trail.

Product outputs:

- Role-specific daily brief acceptance criteria.

Review checkpoint:

- Security and redaction review.

Decision gate:

- Approve internal read-only Command Agent build.

Days 22-30

Engineering outputs:

- Command Agent service.
- AgentCommandPanel.
- Daily Digest or Manager Action Center integration.
- Feedback controls.
- Initial tests and route smoke.

Product outputs:

- Internal pilot plan.
- Feedback taxonomy.

Review checkpoint:

- Internal demo and release-gate review.

Decision gate:

- Launch internal read-only pilot.

Days 31-45

Engineering outputs:

- Cash/Reconciliation Agent read tools.
- Payment reconciliation draft schemas.
- Action draft base model.
- Finance reconciliation assistant panel.

Product outputs:

- Reconciliation triage workflow.
- Match suggestion confidence rules.

Review checkpoint:

- Finance/accounting risk review.

Decision gate:

- Enable read-and-draft for payment/cash pilot.

Days 46-60

Engineering outputs:

- Inventory/Replenishment Agent read tools.
- Replenishment draft schemas.
- Inventory assistant panel.
- Stock-to-cash integration.

Product outputs:

- Replenishment and stock risk pilot workflow.
- Draft language and approval copy.

Review checkpoint:

- Operations/inventory risk review.

Decision gate:

- Enable read-and-draft for inventory pilot.

Days 61-75

Engineering outputs:

- Approval model.
- Approval timeline UI.
- Approval replay validation.
- Idempotency rules.
- Low-risk action execution path.

Product outputs:

- Approval policy matrix.
- Low-risk action list.

Review checkpoint:

- Security and workflow assurance review.

Decision gate:

- Allow low-risk controlled actions only.

Days 76-90

Engineering outputs:

- Agent run admin view.
- Policy incident reporting.
- Cost ledger reporting.
- Observability proof-of-concept.
- Evaluation dataset seed.

Product outputs:

- Pilot scorecard.
- Phase 7 expansion proposal.

Review checkpoint:

- Executive/product/security review.

Decision gate:

- Continue, pause, or expand based on adoption, trust, cost, and safety metrics.

14. Final Recommendation

The first implementation slice should be the shared runtime foundation plus a read-only Command Agent. That slice gives Stoquify the highest leverage because it proves the core pattern without touching dangerous workflows. It also reuses existing Stoquify strengths: protected services, snapshots, business signals, action queues, proof trails, RBAC, module entitlement, redaction, and workflow assurance.

Postpone:

- direct service execution beyond low-risk actions
- statutory/compliance agents
- payroll readiness agent
- close certification workflows
- multi-provider routing
- full MCP exposure
- broad durable execution

Start now with:

- services/agents
- runtime Prisma models

- internal tool registry
- evidence binder
- redaction wrapper
- run logger
- Command Agent panel in Daily Digest or Manager Action Center
- tests and gates proving no direct write authority

This roadmap elevates Stoquify because it turns the application into a trusted daily work system. Stoquify will not merely show records or dashboards. It will explain what matters, prove why it matters, route the right person to the right controlled workflow, and preserve audit-grade evidence along the way.

Immediate Next Implementation Prompt

Use this prompt to begin the first code implementation slice:

Act as a senior Stoquify platform engineer and security architect.

Implement Phase 1 of the Stoquify agent-runtime roadmap only.

Scope:

- Add the shared runtime foundation under services/agents.
- Add the minimum Prisma models needed to record agent definitions, tools, runs, steps, evidence links, feedback, cost ledger, and policy incidents.
- Add a static MVP tool registry for read-only service-owned tools only.
- Add an agent context resolver that uses existing RBAC, module entitlement, tenant, locale, and permission context.
- Add an evidence binder that records source hash, evidence grade, freshness, blockers, redactions, and source modules.
- Add a redaction wrapper that applies existing redaction policies before model context construction.
- Add tests proving tenant isolation, permission filtering, module entitlement filtering, redaction, and no direct Prisma write tool registration.

Non-goals:

- Do not add a visible agent UI yet.
- Do not add AI SDK yet unless absolutely necessary for type contracts.
- Do not add durable execution.
- Do not add observability vendors.
- Do not create action execution tools.
- Do not allow direct Prisma writes, ledger posting, statutory filing, payroll approval, close certification, cash adjustment, stock write-off, or permission changes.

Verification:

- npm run prisma:validate
- npm run typecheck
- npm run lint
- npm run service:boundary:fail
- npm test -- --runInBand services/agents